DevOps Tips/Preparation for Assignment 1 (Part 1 of 2)

This document is designed to give you most of what you need to adapt parts of the Boto 3 example programs provided to complete the Assignment 1 core specification.

Boto 3 Documentation

- To be able to use Boto 3 effectively, you need to be able to browse the documentation. For example:

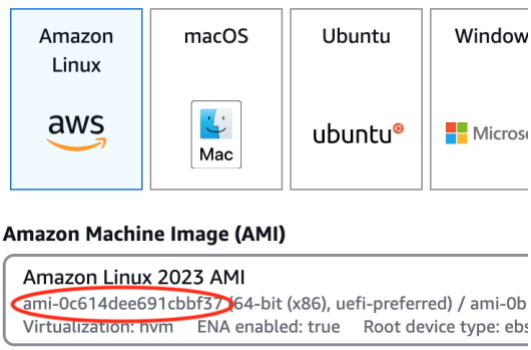
  `ec2 = boto3.resource('ec2')`

  assigns an EC2 *ServiceResource* object to the variable `ec2`. To see what you can do with this object, the following link will take you directly to the documentation for an EC2 service resource:
  https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2.html#service-resource
  As you can see from this documentation, the `create_instances()` method of ServiceResource can take any of a large number of optional parameters, some of which you will want to set to suitable values. For example you'll need to set a `KeyName` for SSH access. You can also set `UserData`, `SecurityGroupIDs` and/or `SecurityGroups` (see below).

Enhanced EC2 instance creation script (Steps 1-3)

One of the main tasks in Assignment 1 is to use Python to create an appropriately configured EC2 instance. A basic *create_instance.py* script was provided in Week 3. Let us look here at how we might enhance this script to make it more useful.

- **Add Key Name**. The `create_instances()` method of the EC2 service has an optional parameter `KeyName`. When specifying a `KeyName` in `create_instances()`, drop the `.pem` extension.
- **Update AMI id**. Make sure that your program is launching an instance from the latest image of Amazon Linux 2023. These are updated from time to time due to security patches, etc. You can find the current AMI id when you start to launch an instance on the web console:



- **Specify Security Group**. When starting a new instance using `create_instances()`, the instance is launched into your VPC's default security group unless you tell it otherwise. You can do this using the `SecurityGroupIDs` or `SecurityGroups` parameter. Each of these parameters is a Python *list* type. To specify a single security group (by ID) which you have already set up, it will look something like this:

```
instances = ec2.create_instances(
    ImageId='ami-0c614dee691cbbf37',
    ...
    SecurityGroupIds=['sg-2a07095e4e3fda0c2'],     # HTTP and SSH
    InstanceType='t2.nano')
```

- **Security groups and VPCs.** Security groups are VPC-specific. When configuring resources (such as security groups) at the management console make sure that you are working in the default VPC (this only becomes a potential issue after the Week 6 lab when you will create a new VPC in your account).

- **Add User Data.** *User data* can be used to configure instance start-up scripts. When setting the `UserData` parameter in `create_instances()`, you will probably want to use a multi-line string. Python allows you to specify a multi-line string by starting and ending it with three quotation marks – i.e. `"""` You can find out more about User Data here: http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html. Note that shell commands provided as user data are run as the *root* user so there is no need to precede with *sudo*.

  For the assignment, your User Data should apply OS patches and install/run the web server. You should test out your user data string by manually launching an instance and entering the user data on the instance configuration screen (expand *Advanced details* and scroll down to the end).

  Instance metadata. EC2 instance metadata is a set of attributes relating to the current instance that are available to programs running on the instance. This data is accessed using a special URL which begins with http://169.254.169.254/latest/meta-data/. The latest version is Instance Metadata Service Version 2 (IMDSv2). This is session-oriented, meaning that you need to start by retrieving a session token which can be valid for up to six hours (21600 seconds). Then you can use this token for instance metadata requests within that time frame.

  In this example we are using instance metadata to display the instance's private IP address:

```
TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-
          metadata-token-ttl-seconds: 21600"`
curl -H "X-aws-ec2-metadata-token: $TOKEN" -s http://169.254.169.254/latest/meta-
          data/local-ipv4
```

  This provides the availability zone in which the instance is located:

```
curl -H "X-aws-ec2-metadata-token: $TOKEN" -s http://169.254.169.254/latest/meta-
          data/placement/availability-zone
```

  Here is an example of User Data code to display some instance metadata on an index page.

```
echo '<html>' > index.html
echo 'Private IP address: ' >> index.html
TOKEN=`curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-
          metadata-token-ttl-seconds: 21600"`
curl -H "X-aws-ec2-metadata-token: $TOKEN"  http://169.254.169.254/latest/meta-
          data/local-ipv4 >> index.html
cp index.html /var/www/html/index.html
```

- **Handle the created instance.** The `create_instances()` method will return a **list** of EC2.Instance objects. This list will usually contain one item, unless you set MinCount and/or MaxCount to values other than 1. You can extract the individual instances using Python list operations – e.g. `instances[0]` is the first instance (i.e. the one instance created when MinCount and MaxCount are both 1).

- An *EC2.Instance* object has several parameters and methods that you can use to manage the instance. See https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/ec2.html#instance . Useful examples include the following:
  - **Reload**. Always call the reload() method on an instance before trying to access its properties – just a simple `instance.reload()` (where the object is named `instance`). This will ensure the object's properties are refreshed.
  - **Tags**. To help keep track of your instances, use the `create_tags()` method of the instance object to add a *Name* tag to the instance; i.e.

```
name_tag = {'Key': 'Name', 'Value': 'Demo instance'}
instance.create_tags(Tags=[name_tag])
```

(where the Instance object is named `instance`). Specifying the *Key* as *Name* ensures that the tag is visible in the default instances view on the AWS management console. Note that tags can alternatively be created at instance creation using `TagSpecifications`, but the above is probably a little easier.

  - You can get public IP address using the `public_ip_address` attribute of the Instance class; i.e. if the object is called `instance`, then it is `instance.public_ip_address`

- **Waiters.** After creating an instance programmatically, you will usually need to wait until the instance reaches the "running" state before doing subsequent tasks. There are a few ways to do this. For example, you could write a loop to keep checking the instance state every few seconds, or (better) you could use a *waiter* method. For example if you have an EC2 instance object called instance, you could do this with:

  ```
  instance.wait_until_running()
  ```

  In conjunction with this, always call the reload() method on an instance before trying to access its properties – just a simple instance.reload() (where the object is named instance). This will ensure the object's properties are refreshed.

  When *wait_until_running()* returns, this does not mean that any User Data script provided has finished – it just means that the instance has reached the "running" state. For example User Data that installs updates and/or a web server may need additional time to complete.

Advice on debugging your program

Build your program incrementally and test after each change. However, be careful to frequently terminate created instances that may build up in number while testing your code repeatedly. Include lots of calls to print() to show yourself what the program is doing – do not "fly blind"!

Note on resources and budget

As well as terminating instances when you no longer need them, make sure you specify *nano* instances at launch.